• Fonksiyon Grupları ⇒ Algoritmaların çalışma süresi fonksiyonlarını büyüme hızlarına göre sınıflandıran matematiksel kategorilerdir.
↳ $c, \log n, n, n\log n, n^2, n^3 \dots \underbrace{2^n}_{NP \text{ problemi}}$

- O, Ω, Θ, o notasyonları ⇒ Asemptotik notasyonlar, bir algoritmanın çalışma süresinin üst sınırını, alt sınırını, tam büyüme sınıfını ve daha sıkı üst sınır ilişkilerini ifade etmek için kullanılır.

* Herhangi algoritmanın karmaşıklığını O olarak söylüyoruz.
Örn; $O(n^2)$ → find / search
↳ Bu fonksiyon için üst sınırı verir. Daha fazlası olmaz.
"Olası üst sınır", ifade eder.
$\Omega(n^2)$
↳ En az n^2 kadar zaman tüketilir. Daha az bekleniti olmaz.
"Olası alt sınır", ifade eder.

$O(n^2)$, $\Omega(n^2)$ > $\Theta(n^2)$ → Tam olarak n^2 kadar bekleniti olur. Ω ve O'nun birleşimidir.

• Genelde O'yu karmaşıklık gösterimi olarak kullanıyoruz.

• Best-case: Yapacağım işi en kolay şekilde yapıyorum. Bir algoritmanın, mümkün olan girdiler arasında en kısa çalışma süresinde çalıştığı durumu, ifade eder.

• Worst-case: Yapılan işi en zor şekilde yapmak. Bir algoritmanın, verilen girdi boyutu için karşılaşabileceği en uzun çalışma süresini ifade eder.

• Kaçta kaç ihtimalle best-case, worst-case gelir gibi analizler hesaplanacak.

• Average-case (Ortalama Durum): Bir algoritmanın, tüm olası girdiler için çalışma sürelerinin olasılıklara göre ortalaması alındığında ortaya çıkan beklenen çalışma süresini ifade eder.

Örnek 1:

-2, 3, 1 ... -6 dizisindeki en küçük elemanı bulma algoritması

yöntem 1:

```
min = i
min_t = -2
for ...
    min_t = item
```

→ if-else vs. yok. 1 döngü var. Yani karmaşıklık → O(n)

→ ilk elemanı alıp başta min olarak onu kabul ediyor.

max ve min'in ikisini de bulan algoritma:

```
max = i
min = i
max_t = -2
min_t = -2
for ...
    if:
        min_t = item
    if:
        max_t = item
```

karmaşıklık → O(n)

yöntem 2:

```
array1.sort()[0]
array1.sort()[ len(array1) - 1 ]
```

O(?) → min

-1 → max

Bu algoritmanın karmaşıklığı sorttan gelen maliyet kadardır. Sort eğer diziyi döndürürse karmaşıklık aynı olur ama bellek ihtiyacı artar.

O(n)

2. Örnek:

-2, 3, 1, ..., -6 dizisindeki 5. büyük eleman nasıl bulunur? Dizinin boyutu maliyetli olarak düşünürsek bunu nasıl çözeriz?

1. yöntem:

array1.sort()[4]

O(?) → sorttan gelen maliyet.

Kitaptaki 2. yöntem:

-2, 3, 1, 4, 7 → ilk 5 elemanı sıraladık.

-2, 3, 1, 4, 7, ~~10~~, ~~15~~, 5, ~~26~~, ...

for → tek 1 döngüde ilk 5 elemanla diğer elemanları karşılaştırıp buluyor.

O(n)

• insertion sort değil ama o mantığı kullanıyor.

Not: array1.sort()

Buradaki sorttaki şu algolar olabilir. Bubble, selection, insertion, bst, heap, quicksort.

3. Örnek:

-2, 3, 1, ..., -6 dizisinde aradığımız elemanı bulma problemi.

search(item)

search(-2): olsun. # True, False

1. yöntem:

```
for...
  :
  if... (aradığım elemanı bulursen)
    :
    :
    break ✗
    continue ✗
    return ✓ → aradığım elemanı bulduğumda return
```

O(n) maliyetli.
1 için O(1)
-6 için O(n)

Search (...) # unsorted

(-2) → ilk eleman → 1 toes' old. ise

* Best-case : $O(1)$, $\frac{1}{n}$ ihtimalle en iyi maliyetle arama sona erer.

(-6) → son eleman

* Worst-case : $O(n)$, $\frac{1}{n}$ ihtimalle en kötü maliyetle arama sona erer.

* average-case : $\frac{n}{2}$ karmaşıklık yani bunu $O(n)$ olarak yazarız.

$-2, 2, 3, \dots, n$

$$\underbrace{1}_{\text{1. de bulma maliyeti (1 for döngüsü)}} + \underbrace{2}_{\text{2. de bulma maliyeti (2 for döngüsü)}} + 3 + \dots + n = \frac{\cancel{n}(n+1)}{2} \Big/ \cancel{n} = \frac{n}{2} \text{ maliyet}$$

average case

n'e bölmemizin nedeni hepsinin maliyetini toplam eleman sayısına bölüp ortalamayı bulmak.

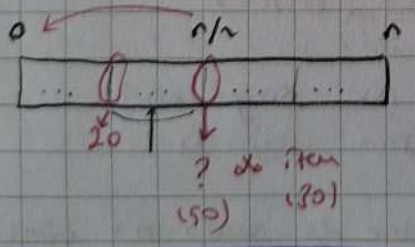
Search (item) # sorted-array
use binary-search

* Dizi bir sıralı dizi olduğunda binary search kullan. Bunun nedeni karmaşık çok fazla olması.

best-case = 1 → aranan eleman tekte bulundu.

worst-case : $\log n$ → Tek bir sayı olunca $\log n$ state'te tek bir sayıya indiriyoruz.

average-case : $\log n$



k tane adım var.

$n \to$ k. eleman
$n/2$
$(n/2)/2$
$\vdots$
3
2
1

$2^k = n$
$k = \log n$

Sıralı dizide orta elemana bakarız. Aradığımla aynı mı? Değilse ve aradığım daha küçükse sol kısmda aramaya başlarız ve dizinin n/2'si gitmiş olur. O da değilse (n/2)/2'si gider.

Bu şekilde devam eder. $\overset{n/2}{(n/2)/2}$
$((n/2)/2)/2$
$\vdots$

} karmaşıklık $O(\log n)$

4. Örnek

-2, 3, 1, ..., 6 dizisini bubble sort ile sırala.

-2, 3, 1, ..., 6

1 3

$\to$ 50 $\to n$

$0 \to n-1$

$0 \to n-2$

$0 \to \dots$

$0 \to 1$

+

$$\frac{n.(n+1)}{2} \to O(n^2)$$

} Bubble sortta karmaşıklık hesaplama maliyeti bu çıkıyor.

Tek bir döngüde buraya 50 geliyor. maliyet n.

Eğer selection sort ile sıralarsak; (İlk indisi sırayla tüm sayılarla karşılaştırıp yerine koyduk)

-2 ... n

3 ... n-1

0 ... n-2

$\vdots$

} Tüm sayılar yerlerine konuldu.

-2'yi yerine koymak için en kötü n tane step gerekir.

0 +

1

$$\frac{n.(n+1)}{2} \to O(n^2)$$

! Best-case'te aradığım sayı ilk sayı mı?
Worst-case'te aradığım sayı son sayı mı?

! Selection ve bubble farkında bubble'da swap az çünkü her sayıyı selection'da karşılaştırıyoruz ama bubble'da 2 sayıyı karşılaştırıyoruz. İkisi de aynı kafalarda / maliyette $O(n^2)$ ama bellek ihtiyaçları (bu swap olayından dolayı) değişiyor.

insertion sort maliyeti

Insertion Sort ne Sıralama: $O(n)$, $O(n^2)$
(Best-case) , (Worst-case)

-2 , 3 , 1 , 4 , 2 , 5 , 6 ... → sonda -10 olsaydı tüm önceki sayılarla karşılaştırılmalıydı.

↳ en sona gelince öncekilerden büyük yerinde kalıyor.

Diziye sıralı gelince tek karşılaştırma gerekiyor.

-2 3 1

-2 1 3 4 2

-2 1 3 2 4

-2 1 2 3 4 5 6

Elemanlar teker teker alınıp önceki sıralı dizi ile karşılaştırılıp orada uygun yere yerleştiriliyor.

↳ Her gelen sayı daha önceki bloktan büyük ise (best-case) bu durumda maliyet:

$$\underbrace{1+1+1+1+\dots+1}_{O(n)} \rightarrow \text{maliyet bu olurdu.}$$

} Best-case

↳ Eğer her seferinde en küçük olan sayı gelirse, en kötü durumda maliyet:

$$\underbrace{\overset{\text{1. eleman}}{1}+2+3+\dots+\overset{\text{15. eleman}}{15}+\dots+\underset{\text{30. eleman}}{30}}_{\frac{n(n+1)}{2} \rightarrow O(n^2)}$$

} worst-case

* En kötü durumda bubble ve selection ile aynı maliyete sahip olur ki bu da düşük ($\frac{1}{n}$) ihtimaldir (worst-case ihtimali).

! Insertion sortta tek bir döngüde veri sıralanırsa $O(n)$ ile buradan çıkarız.

* insertion sort with romenian folk dance.

Bubble → $O(n^2)$
Selection → $O(n^2)$
Insertion → $O(n)$, $O(n^2)$
Merge → $n \log n$
heap → $n \log n$
quick Sort → $n \log n$, n^2

} Yapılar da bazen bellek kadar ihtiyacı oluyor. Duruma göre değişiyor.

! Insertion Sortun hem best-case'i hem worst-case'e göre farklı maaliyetleri var.